

Assignment 1 KBAI

Sem de Vries, S1093742 & Rimski Logtens, S1138380

March 20, 2026

1 Introduction

Artificial intelligence (AI) techniques are increasingly applied to games, both as a benchmark for algorithm development and as a way to study decision-making under uncertainty. Board games, in particular, have long been used as testers for search and heuristic algorithms because they provide a controlled, static environment with easy to follow rules, and clear measurable outcomes.

One well-known example is the game Four-in-a-row. This game is a two player, turn based game where both players try to get four in a row themselves, while preventing the opponent from getting four in a row. The game studied in this project is a game derived from Four-in-a-row, here referred to as N-in-a-row. While the fundamental rules remain unchanged, the environment introduces adjustable parameters, which concern the boards width and height, and the win condition is generalized to achieving N consecutive tokens instead of the standard four. N is also configurable, as long as it is a whole number ≥ 1 .

This project investigates the implementation and comparison of two classical game tree search algorithms: **Minimax** and **Alpha-Beta pruning**. The aim is to evaluate their efficiency when applied to the N-in-a-row game environment. A key aspect of this study is the choice of runtime measure. Rather than relying on measures depending on timing (e.g., **ms**), the number of heuristic evaluations performed per game is used. This is done, as Alpha-Beta pruning should cut off branches when certain conditions are met, thus resulting in less heuristic evaluations per game.

2 Methods

2.1 Design

The goal of the N-in-a-row game is to achieve N consecutive tokens in a row. This can be horizontally, vertically, and diagonally. Therefore all 3 axis need to be observed and checked. This proposes the problem that 3 different states need to be checked by the player or the algorithm to see which option is the most optimal.

2.2 Used Algorithms

Algorithms used are the MiniMax algorithm and the extension of this algorithm Alpha-Beta pruning. Below the Min Max algorithm pseudocode is shown:

Algorithm 1 Minimax Algorithm

```
1: function MINIMAX(node, depth, maximizingPlayer)
2:   if depth = 0 or node is a terminal node then
3:     return the heuristic value of node
4:   end if
5:   if maximizingPlayer then
6:      $maxEval \leftarrow -\infty$ 
7:     for each child of node do
8:        $eval \leftarrow \text{MINIMAX}(child, depth - 1, \text{false})$ 
9:        $maxEval \leftarrow \text{MAX}(maxEval, eval)$ 
10:    end for
11:    return  $maxEval$ 
12:   else
13:      $minEval \leftarrow +\infty$ 
14:     for each child of node do
15:        $eval \leftarrow \text{MINIMAX}(child, depth - 1, \text{true})$ 
16:        $minEval \leftarrow \text{MIN}(minEval, eval)$ 
17:     end for
18:     return  $minEval$ 
19:   end if
20: end function
```

2.3 Explanation of Minimax Algorithm

The fundamental game tree search strategy used is the Minimax algorithm, along with its optimized extension, Alpha-Beta pruning. The Minimax function, Minimax, operates recursively, first checking a base condition: if the maximum search depth is reached or the current node is terminal, it returns its heuristic

value. The algorithm then distinguishes between the two players: for the maximizingPlayer, it initializes maxEval to $-\infty$ and recursively evaluates all child nodes, updating maxEval with the maximum returned value; conversely, for the minimizingPlayer, it initializes minEval to $+\infty$ and updates minEval with the minimum returned value. By exploring the entire game tree up to a specified depth and assuming optimal play from the opponent, the function ultimately returns the best achievable value for the current player.

2.4 Game Playing Algorithms

Here is the pseudocode for the Alpha-Beta Pruning algorithm (see Algorithm 2). This algorithm optimizes the Minimax search by pruning branches that cannot influence the final outcome. Below the Alpha beta pruning algorithm pseudocode is shown:

Algorithm 2 Alpha-Beta Pruning

```

1: function ALPHABETA(node, depth,  $\alpha$ ,  $\beta$ , isMaximizingPlayer)
2:   if depth = 0 or node is a terminal node then
3:     return the heuristic value of node
4:   end if
5:   if isMaximizingPlayer then
6:     value  $\leftarrow -\infty$ 
7:     for each child of node do
8:       eval  $\leftarrow$  ALPHABETA(child, depth - 1,  $\alpha$ ,  $\beta$ , false)
9:       value  $\leftarrow$  MAX(value, eval)
10:       $\alpha \leftarrow$  MAX( $\alpha$ , value)
11:       $\alpha \geq \beta$ 
12:      break ▷ Beta cut-off
13:
14:     end for
15:     return value
16:   else ▷ Minimizing player
17:     value  $\leftarrow +\infty$ 
18:     for each child of node do
19:       eval  $\leftarrow$  ALPHABETA(child, depth - 1,  $\alpha$ ,  $\beta$ , true)
20:       value  $\leftarrow$  MIN(value, eval)
21:        $\beta \leftarrow$  MIN( $\beta$ , value)
22:        $\beta \leq \alpha$ 
23:       break ▷ Alpha cut-off
24:
25:     end for
26:     return value
27:   end if
28: end function

```

2.5 Explanation of the Pseudocode for Alpha-Beta Pruning.

The Alpha-Beta Pruning algorithm operates as a recursive search function, Alpha-Beta, designed to optimize decision-making in two-player games by efficiently exploring the game tree. The recursion establishes a base case where the heuristic value of a node is returned if the maximum search depth is reached or if the node is terminal. The algorithm then bifurcates based on whether the current state corresponds to the Maximizing Player (Minimax Max layer) or the Minimizing Player (Minimax Min layer). The Maximizing Player initializes the value to $-\infty$ and updates the α parameter (the best value found so far for Max) after each recursive call; if α becomes greater than or equal to the β parameter (the best value found so far for Min), a Beta cut-off occurs, and the loop breaks. Conversely, the Minimizing Player initializes the value to $+\infty$ and updates the β parameter; if β becomes less than or equal to α , an Alpha cut-off occurs. These cut-offs represent the core optimization, pruning branches of the game tree that are guaranteed not to affect the final decision, with the function ultimately returning the final optimal value for the current node.

2.6 Goal of the algorithm

The Minimax algorithm and the Alpha Beta pruning algorithm were used to optimize the AI guided player to somewhat give resistance against the human player. By tweaking the board and or parameters of the AI algorithms the outcome of the game could be altered. With the most optimal values making it harder for the human player to win.

3 Results

3.1 Test Table

All three tests were completed without issue. The results of these test are depicted in the table down below. For the runs with a different depth each run was done with a standard board size of 7x6 and $N = 4$. The variable depth was changed each run to clearly visualize the performance difference between the two algorithms. We made the same moves in all games, to ensure the best possible comparison. Unfortunately, this does not always provide the same game, so we will always evaluate the **average evaluations** (AE), which represents $\frac{\text{evaluations}}{\text{moves}}$. For run number 2 the depth was set at 3 for all trials, with the $N=4$ with the changing factor being a different board size. For the third run the board size was kept the same at 7x6 and the depth was kept the same at 3, but the changing factor here was the amount of N needed to win the in a row game.

	Trial 1		Trial 2		Trial 3	
	Depth	AE	Depth	AE	Depth	AE
MiniMax	3	315.2	5	13569.67	7	643087.86
Alpha-Beta pruning	3	125.33	5	1119	7	16161.29

Table 1: Depth and Average Evaluations per Algorithm

	Trial 1		Trial 2		Trial 3	
	Boardsize	AE	Boardsize	AE	Boardsize	AE
MiniMax	5x5	97	7x7	296.4285714	9x9	658.75
Alpha-Beta pruning	5x5	47.66666667	7x7	118.7142857	9x9	211.125

Table 2: Changing The Board Size and Average Evaluations per Algorithm

	Trial 1		Trial 2		Trial 3	
	N-value	AE	N-value	AE	N-value	AE
MiniMax	3	272.2	4	182.7142857	5	213.6666667
Alpha-Beta pruning	3	86.33333333	4	59.17647059	5	72

Table 3: Changing N Size to Win and Average Evaluations per Algorithm

3.2 Runtime Complexity

In addition to average evaluation counts, it is important to consider the theoretical complexity of the algorithms. Minimax explores the entire game tree with complexity $O(b^d)$, where b is the branching factor and d is the depth of the search. This exponential growth makes the algorithm computationally expensive at higher depths, which is shown in Table 1.

Alpha-Beta pruning improves efficiency by eliminating branches of the tree that cannot affect the final decision (Wikipedia, 2025)(1). Although its worst-case complexity is the same as Minimax ($O(b^d)$), in the best case, with optimal move ordering, the effective branching factor is reduced to approximately $\sqrt{b}$. This gives a runtime complexity of $O(b^{\frac{d}{2}})$, which is substantial improvement over Minimax (IRJET, 2018)(2).

These theoretical results align with the empirical findings in this project, where Alpha-Beta pruning, on average, requires fewer heuristic evaluation than Minimax at equivalent depths, different n-sizes to win and different board sizes.

4 Discussion

Both the Minimax and Alpha-Beta pruning algorithms worked well, as has been demonstrated in the test runs. However there were some complications, in the performance of the algorithms. Especially later on in the games and when complicated situations arose such as multiple 3 in a rows when 4 in a row was the winning state. It can be concluded from the results section that Alpha-Beta pruning significantly reduces the amount of board evaluations compared to the MiniMax algorithm. This aligns with the theoretical expectations that pruning reduces the effective branching factor of the search tree. The advantage of using less board evaluations is that less computational power is necessary to achieve the same result. This therefore improves the loading speed after each run, therefore improving the user experience.

5 Conclusion

To conclude it was found in the research that the use of the MiniMax algorithm useful with Alpha beta pruning significantly improving the amount of board evaluations necessary to achieve the same result. Therefore Alpha-Beta pruning can be seen as the most efficient algorithm to play the n in a row game. As for collaboration, this was sufficient with both Sem and Rinski taking on half of the programming and writing work each.

5.1 Video conclusion

The youtube video used to compile the code can be found here (3)

6 References

References

- [1] Wikipedia. (2025). *Alpha-beta pruning*. In *Wikipedia, The Free Encyclopedia*. Retrieved November 13, 2025, from https://en.wikipedia.org/wiki/Alpha%E2%80%93beta_pruning
- [2] Nasa, R., & colleagues. (2018). *Alpha-Beta Pruning in Mini-Max Algorithm – An Optimized Approach for a Connect-4 Game*. International Research Journal of Engineering and Technology (IRJET), 5(4), 1637–1641. <https://www.irjet.net/archives/V5/i4/IRJET-V5I4366.pdf>
- [3] Lague, S. (2018). *Algorithms Explained – Minimax and Alpha-Beta Pruning* [YouTube video]. YouTube. <https://www.youtube.com/watch?v=1-hh51ncgDI>